

1.API 직접 호출 방식과 MCP 기반 Tool 호출

API 직접 호출 방식의 장점과 한계

API 직접 호출 방식은 필요한 기능을 빠르게 연결하고 결과를 확인하기에 유리합니다. PoC 초기에는 특정 DB 조회 함수나 문서 검색 함수를 코드에서 직접 호출하는 방식이 단순하고, 동작 여부를 빠르게 검증할 수 있습니다. 예를 들어 설비 상태를 조회하는 함수, 알람 이력을 가져오는 함수, 매뉴얼을 검색하는 함수를 각각 코드에서 바로 호출하면 구현 속도는 빠릅니다.

직접 호출 방식의 장점

- 구현 속도가 빠르고 초기 동작 검증에 유리
- 코드 구조가 단순하여 PoC 초기에 적합
- 특정 함수 단위로 즉시 테스트 가능
- 별도 계층 없이 결과를 바로 확인 가능

호출 대상이 늘어날 때의 구조적 한계

호출 로직이 여러 파일에 흩어지고, 어떤 함수가 어떤 권한과 입력값으로 호출되는지 추적하기 어려워질 수 있습니다. 오류 처리 방식이 함수마다 다르면 실패 원인 분석도 어려워지고, State, agent_steps, 결과 파일 또는 운영 Trace로 남겨야 할 실행 기록 기준도 일관되지 않습니다.

제조 현업 Agent에서는 DB, Log, 기술 문서, 품질 지표처럼 호출 대상이 많기 때문에 단순 직접 호출만으로는 운영과 검증이 어려워질 수 있습니다. 따라서 **API 직접 호출**은 빠른 확인에는 적합하지만, 운영 가능한 Agent 구조로 확장하려면 표준화된 Tool 계층이 필요합니다.

 오늘은 설계 관점에서 봅니다. 직접 호출 방식이 왜 PoC 이후 단계에서 한계를 드러내는지, 그리고 표준화된 Tool 계층이 왜 필요한지를 제조 현업 맥락에서 이해하는 것이 이 슬라이드의 목적입니다.

MCP 기반 Tool 호출 방식의 의미

MCP 기반 Tool 호출은 Agent가 외부 기능을 임의로 호출하는 것이 아니라, 정해진 Tool 규격을 통해 호출하게 하는 구조입니다. 여기서 Tool은 개발자가 사용하는 일반적인 도구가 아니라, Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위입니다. 예를 들어 설비 상태 조회, 알람 이력 확인, 품질 지표 조회, 기술 문서 검색 같은 기능을 Agent Tool로 볼 수 있습니다.

Tool 표준화의 핵심 요소

Tool 이름, 설명, 입력, 출력, 오류 처리 기준을 표준화하면 LLM과 Agent가 어떤 기능을 언제 사용해야 하는지 더 명확히 판단할 수 있습니다. 이는 단순한 코드 정리가 아니라 Agent의 판단 기준을 설계하는 작업입니다.

내부 시스템 보호 계층으로서의 Tool

내부 시스템을 Agent가 직접 다루지 않고 Tool 계층 뒤에 두면, DB 구조나 API 구현이 바뀌어도 Agent의 상위 흐름은 비교적 안정적으로 유지할 수 있습니다. 내부 시스템을 Agent Tool로 감싸는 구조를 설계하는 것이 핵심입니다.

계약 구조로서의 MCP Tool

MCP Tool은 단순 함수 호출을 감싸는 것이 아니라, 호출 규칙과 결과 해석 방식을 함께 정리하는 계약 구조로 이해해야 합니다. Tool 이름과 설명은 LLM이 Tool을 선택하는 기준이 되고, 입출력 구조는 Agent가 결과를 해석하는 방식을 결정합니다.

실행 기록과 평가 연결

State, agent_steps, 결과 파일 또는 운영 Trace 형태로 실행 기록을 함께 남기면 어떤 Tool이 어떤 입력으로 호출되었고, 어떤 결과와 오류가 발생했는지 이후 평가와 개선에 활용할 수 있습니다. 이는 4일차 평가와 Guardrail 검증의 근거가 됩니다.

📌 3일차에서는 MCP의 세부 구현 문법보다 내부 시스템 기능을 Agent가 호출 가능한 표준 Tool로 정리하는 설계 관점에 집중합니다.

제조 현업에서 MCP가 필요한 이유

제조 현업에서는 하나의 질문에 답하기 위해 여러 종류의 정보가 함께 필요할 수 있습니다. 예를 들어 설비 알람 원인을 확인하려면 알람 이력, 설비 상태, 공정 상태, 품질 지표, 정비 이력, 기술 문서 검색 결과가 함께 사용될 수 있습니다.

직접 호출 방식의 분산 문제

이 기능들을 각각 직접 호출 방식으로 붙이면 빠르게 동작은 만들 수 있지만, 권한 관리, 반환 데이터 범위, 오류 처리, 감사 로그 기준이 여러 위치로 분산될 수 있습니다. 호출 대상이 많아질수록 이 분산 문제는 더욱 심각해집니다.

MCP Tool 구조의 해결 방식

MCP 기반 Tool 구조는 이러한 기능들을 하나의 표준 호출 방식으로 정리하여 Agent가 일관된 방식으로 사용할 수 있게 합니다. Tool별로 어떤 입력을 받을지, 어떤 데이터를 반환할지, 오류가 발생했을 때 어떤 형식으로 남길지 정의하면 사내 PoC에서 운영 구조로 확장하기 쉬워집니다.

LangGraph 및 Multi-Agent 연결

Tool 이름, 설명, 입력·출력 구조가 정리되어 있어야 LangGraph Routing에서 어떤 조건으로 어떤 Tool을 호출할지 판단할 수 있습니다. Multi-Agent 구조에서도 Agent별 호출 책임과 State 전달 범위를 명확히 할 수 있습니다.

4일차 평가와의 연결

4일차에는 이 구조를 기준으로 Tool 호출이 적절했는지, State와 agent_steps, 결과 파일에 실행 흐름이 충분히 남았는지, Guardrail이 필요한 지점이 어디인지 평가할 수 있습니다. Tool 구조가 명확할수록 평가 기준도 명확해집니다.

API 직접 호출과 MCP 기반 Tool 호출을 구분하는 기준

이 장은 API 직접 호출 방식과 MCP 기반 Tool 호출 방식을 비교하여, 3일차에서 왜 Tool 설계가 중요한지 정리하는 장입니다.

비교 항목	API 직접 호출	MCP 기반 Tool 호출
구현 속도	빠름 — PoC 초기에 유리	설계 시간 필요, 운영 확장에 유리
권한 관리	함수마다 분산될 수 있음	Tool 단위로 표준화 가능
오류 처리	함수마다 방식이 다를 수 있음	Tool 규격으로 일관된 처리 가능
실행 기록	State/Trace 기준이 흩어질 수 있음	agent_steps, 결과 파일로 일관 기록
LLM 판단 기준	명확한 선택 기준 없음	Tool 이름·설명이 선택 기준이 됨
유지보수	호출 대상 증가 시 복잡도 증가	Tool 계층 분리로 상위 흐름 안정

제조 현업에서는 DB, Log, 문서, API가 여러 계층에 존재하므로 단순 호출보다 Tool 이름·설명·입출력 구조와 실행 기록 기준이 중요합니다. 3일차 실습에서는 API 직접 호출과 MCP 기반 호출의 차이를 이해한 뒤, DB/Log Tool과 RAG Tool을 어떤 규격으로 정리할지 살펴봅니다.

 다음 장에서는 실제로 MCP Tool을 어떤 기준으로 나누어야 하는지, 업무 단위·데이터 소스 단위·권한 단위·위험도 단위로 구체적으로 검토합니다.

2.Tool 분할

Tool 분할이 중요한 이유

MCP Tool을 나누는 일은 단순히 함수를 여러 개 만드는 작업이 아닙니다. MCP는 Agent가 외부 기능을 호출할 때 사용하는 표준 연결 방식으로 이해할 수 있으며, 여기서는 제조 DB, 설비 로그, 기술 문서 검색 같은 기능을 Agent가 일관된 규격으로 호출하게 만드는 구조로 다룹니다.

Tool은 Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위입니다. 예를 들어 설비 상태 조회, 알람 이력 확인, 품질 지표 조회, 기술 문서 검색 같은 기능을 Agent Tool로 볼 수 있습니다. Tool은 LLM과 Agent가 외부 시스템 기능을 선택하고 호출하는 기준이 됩니다.

Tool이 너무 클 때

하나의 Tool 안에 여러 업무가 섞여 역할이 모호해지고, LLM이 언제 사용해야 하는지 판단하기 어려워집니다. 설비 상태, 알람 이력, 품질 지표, 기술 문서 근거가 뒤섞일 수 있습니다.

Tool이 너무 작을 때

호출 횟수가 늘어나고, LangGraph Routing 흐름과 Multi-Agent 간 State 전달이 복잡해집니다. 불필요한 호출이 반복되어 성능과 추적 모두 어려워집니다.

균형 있는 분할의 기준

Tool 분할은 코드 구조 문제가 아니라 업무 목적, 데이터 성격, 권한, 위험도를 함께 고려하는 아키텍처 설계 문제로 보아야 합니다.

예를 들어 설비 반복 알람 원인을 확인하는 Agent가 하나의 거대한 Tool만 호출하도록 설계되면 설비 상태, 알람 이력, 품질 지표, 기술 문서 근거가 뒤섞일 수 있습니다. LLM이 Tool을 잘 선택할 수 있도록 역할과 사용 범위를 명확히 하는 것이 Tool 분할의 핵심 목적입니다.

❏ Tool 분할은 코드 문제가 아니라 업무 설계 문제입니다. 이 관점을 유지하면서 다음 슬라이드의 구체적인 분할 기준을 살펴봅니다.

업무 단위와 데이터 소스 단위 분리

제조 Agent는 하나의 질문에 답하기 위해 여러 종류의 정보를 함께 사용할 수 있습니다. 예를 들어 설비 알람 원인을 확인하려면 설비 기본 정보, 최근 알람 이력, 품질 지표 변화, 정비 이력, 기술 문서 근거가 함께 필요할 수 있습니다. 이때 모든 조회를 하나의 Tool에 넣으면 반환 데이터가 과도해지고, Tool의 목적도 불명확해집니다. Agent가 어떤 근거를 사용해 판단했는지 추적하기도 어려워집니다.

업무 단위 분리 예시

- 설비 상태 조회 — 기본 정보, 위치, 공정 정보
- 알람 이력 조회 — 반복 발생 여부, 알람 유형
- 품질 지표 조회 — 불량률, 수율 변화
- 정비 이력 조회 — 최근 정비, 부품 교체 이력

각 업무 단위는 하나의 명확한 목적을 가지며, LLM이 어떤 상황에서 어떤 Tool을 선택해야 하는지 판단할 수 있는 기준이 됩니다.

데이터 소스 단위 분리: DB Tool vs RAG Tool

DB Tool은 실제 발생한 상태, 이력, 수치 데이터를 확인하는 데 적합합니다. 제조 DB에서 조회한 알람 이력은 실제 발생 데이터이며, 수치와 시간 기반의 사실 정보를 다룹니다.

RAG Tool은 기술 문서, 장애 대응 가이드, 조치 절차처럼 문서 근거를 검색하는 데 적합합니다. 기술 문서 검색으로 찾은 조치 절차는 문서 근거이며, 사실 데이터와 성격이 다릅니다.

이처럼 근거의 성격이 다른 기능을 분리하면 Agent가 어떤 정보가 사실 데이터이고 어떤 정보가 문서 근거인지 더 명확히 다룰 수 있습니다. State의 db_results와 rag_results를 구분하는 것도 이 원칙에서 비롯됩니다.

권한 단위와 위험도 단위 분리

제조 현업 Agent에서 모든 Tool을 같은 수준으로 다루면 안 됩니다. 단순 조회 Tool과 민감 정보가 포함될 수 있는 Tool, 실제 조치나 변경을 수행하는 Tool은 권한과 위험도가 다릅니다.

Read-Only Tool

설비 기본 정보 확인, 알람 이력 조회, 품질 지표 확인처럼 데이터를 읽기만 하는 기능입니다. 일반적인 조회 권한으로 호출 가능하며, 실제 업무 행동을 유발하지 않습니다. 반복 알람 분석 중 설비 상태와 최근 알람 이력을 확인하는 단계는 조회 중심으로 처리할 수 있습니다.

Action Tool

생산 조건 변경, 정비 요청 생성, 작업 지시 등록처럼 실제 업무 행동을 유발하는 기능입니다. 조치 요청이나 작업 지시와 연결되는 단계는 별도 Tool로 분리하고 Guardrail 또는 Human Review를 거치도록 설계하는 것이 안전합니다.

민감 정보나 과도한 조회 가능성이 있는 Tool은 입력 조건, 반환 필드, 조회 범위, 승인 절차를 더 엄격하게 설계해야 합니다. 이러한 분리는 단순 보안 조치가 아니라, Agent가 잘못된 판단으로 위험한 행동을 하지 않도록 만드는 운영 설계 기준입니다.



따라서 read-only Tool과 action Tool은 반드시 분리하고, 권한과 위험도에 따라 호출 가능 조건을 다르게 설계해야 합니다. 이 기준은 4일차 Guardrail 설계와 직접 연결됩니다.

실습 예제 기준 Tool 분할표

3일차 실습 예제에서는 제조 장애 대응 흐름을 이해하기 위해 여러 Tool을 업무 목적별로 나누어 사용합니다. 각 Tool이 "무엇을 하는지"뿐 아니라 "무엇을 하지 않는지"도 명확히 해야 한다는 점이 중요합니다.

Tool 이름	유형	주요 목적	하지 않는 것
get_equipment_status	DB Tool	설비 기본 정보, 위치, 공정 정보 확인	알람 이력 조회, 조치 절차 설명 아님
get_recent_alarm_events	DB Tool	특정 설비·알람 유형의 반복 발생 여부 확인	조치 절차 설명, 문서 검색 아님
get_quality_metrics	DB Tool	품질 지표 변화, 불량률, 수율 변화 확인	알람 원인 분석, 정비 이력 조회 아님
get_maintenance_history	DB Tool	최근 정비 이력, 부품 교체 이력 확인	기술 문서 검색, 조치 지시 아님
search_manual	RAG Tool	기술 문서, 장애 대응 가이드, 조치 절차 검색	실제 알람 발생 이력 조회 아님

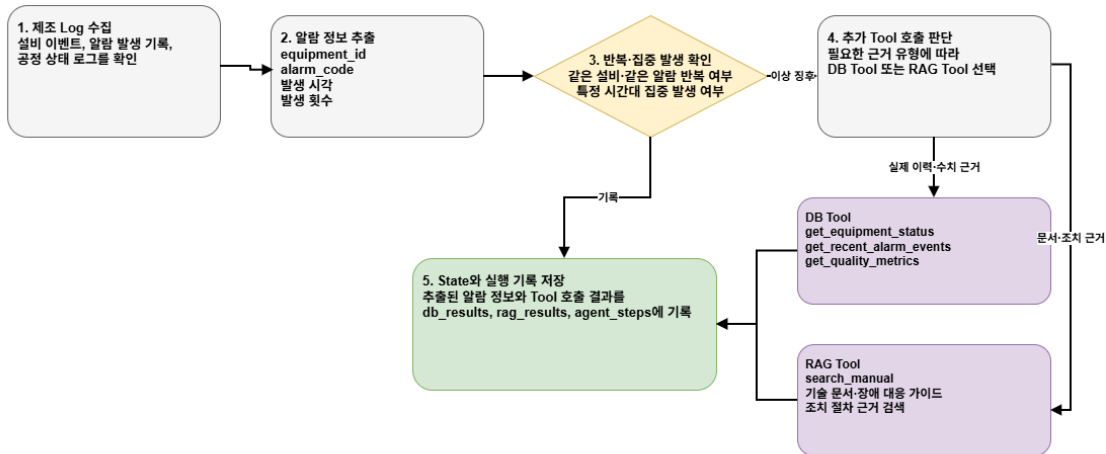
반복 알람 분석 흐름에서는 설비 상태를 확인한 뒤 알람 이력을 조회하고, 품질 지표와 정비 이력을 함께 검토한 다음, 필요한 경우 기술 문서 검색으로 조치 절차의 근거를 보완하는 식으로 각 Tool의 목적을 분리해 사용할 수 있습니다.

 실습 파일 연결: postgres_db_tool.py (DB/Log Tool), search_manual_tool.py (RAG Tool wrapper), manufacturing_mcp_server.py (FastMCP Tool Server)

제조 Log 기반 이상 징후 확인 흐름

제조 Log는 Agent가 직접 원문 전체를 해석하는 대상이 아니라, Tool 계층을 통해 필요한 알람 정보만 추출되어야 하는 내부 데이터입니다. Agent는 Log 원문을 그대로 다루기보다 설비 ID, 알람 코드, 발생 시각, 반복 발생 여부처럼 판단에 필요한 정보만 Tool을 통해 받아야 합니다.

제조 Log 기반 이상 징후 확인 흐름



정리
반복·집중 알람 확인은 단순 로그 조회가 아니라,
이후 DB Tool, RAG Tool, LangGraph Routing으로 이어지는 판단 출발점입니다.

현업 적용 시 Tool 분할 체크리스트

현업에 MCP Tool 구조를 적용할 때는 먼저 Tool이 하나의 명확한 업무 목적을 갖는지 확인해야 합니다. 다른 Tool과 역할이 겹치면 LLM이 어떤 Tool을 선택해야 할지 혼동할 수 있고, 같은 질문에 대해 불필요한 중복 호출이 발생할 수 있습니다.

Tool 역할 명확성 확인

이 Tool이 하나의 명확한 업무 목적을 갖는가? 다른 Tool과 역할이 겹치지 않는가? LLM이 어떤 상황에서 이 Tool을 선택해야 하는지 설명할 수 있는가?

입력 조건과 반환 범위 제한

입력값이 너무 자유롭거나 제한이 없으면 과도한 조회, 잘못된 조회, 권한 범위 초과 문제가 생길 수 있습니다. 반환 데이터가 지나치게 많으면 Agent가 불필요한 정보를 처리하게 되고, 민감정보 노출 위험도 커집니다.

1

권한 및 승인 절차 정의

사내 PoC로 전환할 때 제조 DB 조회 Tool은 조회 가능한 설비 범위와 반환 필드를 제한해야 하고, 조치 요청 Tool은 승인 절차와 Human Review 기준을 포함해야 합니다.

2

실행 기록 항목 정의

기술 문서 검색 Tool은 검색 대상 문서 범위와 근거 출처를 남겨야 합니다. State, agent_steps, db_results, rag_results, 결과 JSON/Markdown 중 어떤 항목을 실행 기록으로 남길지 Tool별로 정의해야 합니다.



이 체크리스트는 Tool을 많이 만드는 것이 아니라, Agent가 안정적으로 선택하고 호출할 수 있는 Tool을 설계하기 위한 기준입니다. 교육용 구조를 실제 현업 구조로 어떻게 대체할지 판단하는 데 활용하십시오.

5장 정리: MCP Tool 분할 기준을 현업에 적용하기

이 장은 MCP Tool을 어떤 기준으로 나눌지 정리하는 장입니다. Tool 분할은 함수나 파일을 나누는 문제가 아니라, 제조 업무 기능과 데이터 접근 범위를 어떻게 설계할지 결정하는 문제입니다.

분할 기준 요약

- 업무 단위: DB 근거와 문서 근거를 명확히 분리
- 데이터 소스 단위: DB Tool과 RAG Tool 구분
- 권한 단위: 조회와 조치 실행을 안전하게 분리
- 위험도 단위: read-only Tool과 action Tool 분리

Tool은 너무 커도 역할이 모호해지고, 너무 작아도 호출 수와 Routing 복잡도가 증가하므로 균형 있게 설계해야 합니다.

실습 예제 Tool과 현업 전환 판단

실습 예제의 `get_equipment_status`, `get_recent_alarm_events`, `get_quality_metrics`, `get_maintenance_history`, `search_manual`은 이러한 분할 기준을 이해하기 위한 교육용 예시입니다.

특히 `get_equipment_overview`는 여러 정보를 한 번에 조회하는 통합 Tool이므로, 편리성은 있지만 Tool 역할이 커질 수 있다는 점을 함께 검토해야 합니다.

반복 알람 분석 Agent를 사내 PoC로 확장한다면 기존 조회 Tool을 유지할지, 품질 지표나 정비 이력을 별도 Tool로 분리할지, 조치 요청 기능을 action Tool로 새로 설계할지 판단해야 합니다.

 다음 장에서는 이렇게 나눈 Tool을 Agent가 일관되게 사용할 수 있도록 Tool Contract와 Tool Description을 어떻게 설계해야 하는지 살펴봅니다. Tool 이름, 설명, 입력·출력 구조, 권한, 반환 데이터 범위, 실행 기록 항목을 함께 검토하는 것이 핵심입니다.